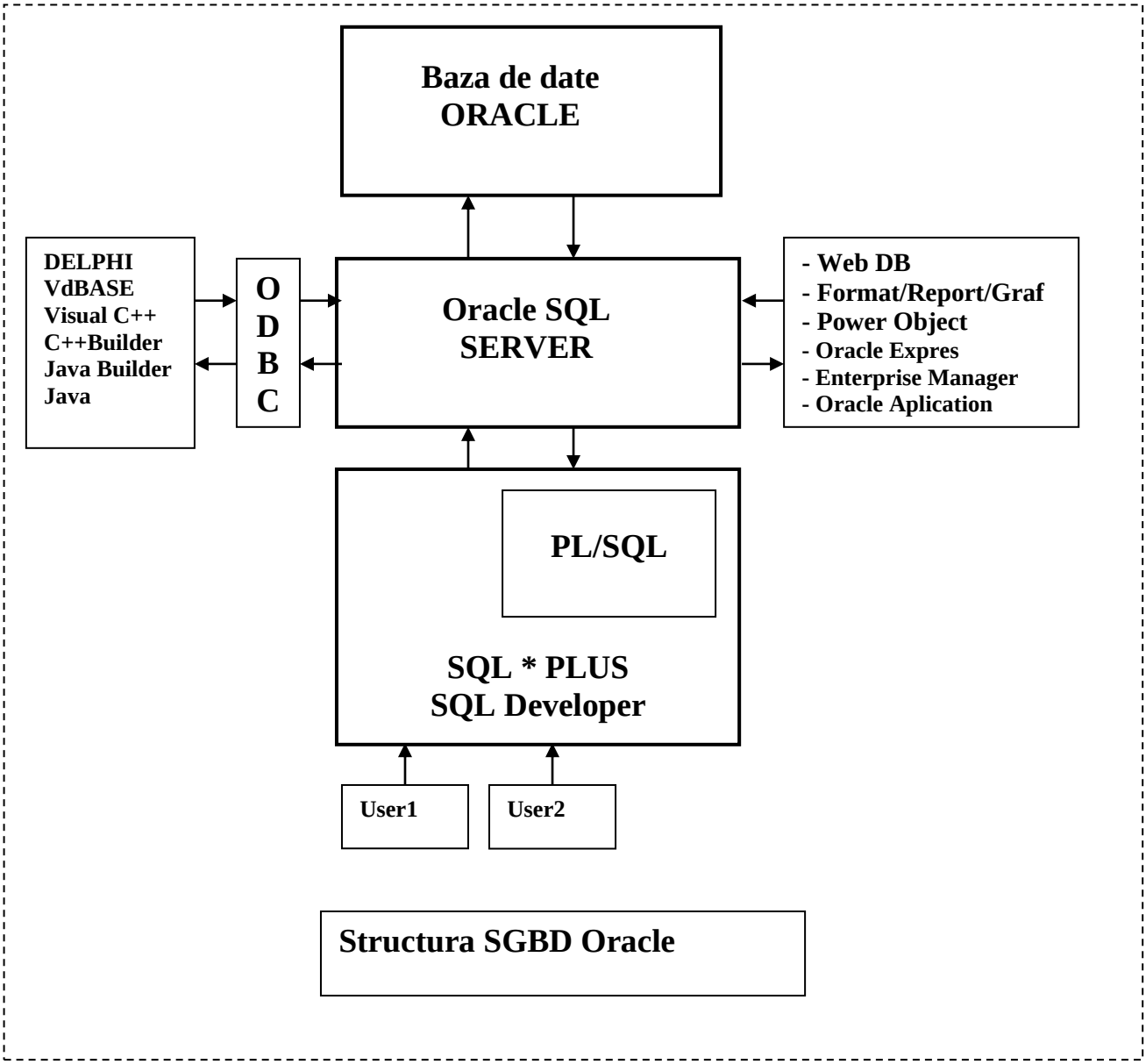
1. COMENZI SQL IN ORACLE 8I, 9I, 10G

Documentul reprezinta un extras din: I. Jian, Proiectarea Bazelor de Date – Material de curs

1.1. Structura SGBD Oracle



Componentele de baza Oracle sunt:

- Oracle SQL Server - executa toate comenzile SQL
- SQL*Plus - mediu de dezvoltare, interfaţa cu utilizatorii
- PL/SQL - Limbaj procedural pentru dezvoltare aplicaţii

Oracle SQL Server este singurul component Oracle care are acces la informaţiile din Baza de Date, asigurând integritatea si securitatea informaţiilor memorate. Serverul execută toate comenzile SQL pentru toţi utilizatorii conectaţi la staţii. Pentru fiecare utilizator conectat se deschide o sesiune separată. Fiecare utilizator va avea parola lui, dar mai multe sesiuni se pot lansa pe acelaşi user de la staţii diferite sau de pe aceeaşi staţie.
Toată interfaţa cu utilizatorii este asigurată de către SQL*Plus, care are unele comenzi de editare, permite comunicarea cu Qracle SQL Server si lansează programe PL/SQL.
Pe o statie se pot lansa mai multe sesiuni prin SQL+, chiar pe servere diferite.

Lansarea SQL*Plus se face printr-un shortcut sau:

START | Programs | Oracle OraHome.. | Aplication Developement | SQL*Plus

In fereastra deschisa pentru conectare se completează:
User Name: **SCOTT** *tab* - user existent cu drepturi limitate
Password: **TIGER** *tab* - parola utilizatorului
Host string: **B623server** *enter* - Numele Bazei de Date existente pe server
Pentru sala B529 la numele bazei de date (host) se da:
Adresa IP a serverului Oracle **192.168.113.200:1522/orcl**
192.16.21.100:1521/orcl2

1.2. Comenzi SQL

Crearea unei tabele se face precizând intr-o lista numele şi tipul câmpurilor.

CREATE TABLE Stud (Nume Char(20), Adresa Varchar2(15),Bursa Number(7),Datan Date);
Table created.

Tipul Varchar2(20) este şir de caractere de lungime variabila maxim 20 caractere.

Ştergerea unei tabele**.**

SQL> **DROP TABLE** stud1; -- nu salvează înregistrările in Rollback
Table dropped.

Adaugarea de inregistrari intr-o tabela:

INSERT INTO stud1 (nume, adresa, bursa, datan**)** - lista de câmpuri
VALUES ('Radu', 'Lugoj', 1200,'15-jan-77'); - lista de valori
1 row created.

Stergere inregistrări:

DELETE FROM stud **WHERE** Adresa Like ‘Lugoj%’;
% indica un numar de caractere oarecare
_ indica un caracter care poate avea orice valoare

Atenţie!! DELETE FROM stud; -- şterge toate înregistrările salvându-le în Rollback

ROLLBACK; -- reface toate inregistrarile sterse sau modificatedupă ultimul COMMIT

COMMIT; -- şterge fişierul Rollback si modificările sunt definitive

TRUNCATE TABLE Stud; --şterge numai înregistrările si tabela rămâne goala

Modificarea conţinutului unor înregistrări se face folosind comanda **Update**

UPDATE stud set bursa=1200 WHERE Nume= 'Radu';

UPDATE stud set bursa=bursa*1.2; -- mareste toate bursele cu 20%

Afisarea tuturor tabelelor din spatiul utilizator:

SELECT * FROM TAB; - afişare numele tabelelor utilizatorului curent
SELECT * FROM CAT; - afişare catalog utilizator

```
TABLE_NAME                     TABLE_TYPE
------------------------------ -----------
ADRESE                         TABLE
ANGAJATI                       TABLE
BONUS                          TABLE
DEPART                         TABLE
DEPT                           TABLE
DISCIPLINE                     TABLE
DISCSPECPROF                   TABLE
EMP                            TABLE
EMP1                           TABLE
EMP2                           TABLE
NOTE                           TABLE
```

SELECT Owner,table_name from **All_tables**; - tabela cu toate datele despre tabele

Fiind de ordinul sutelor, se vor afişa numai tabele care aparţin utilizatorului SCOTT.

SELECT Owner,table_name from All_tables where owner='SCOTT'

```
OWNER                          TABLE_NAME
------------------------------ ------------------------------
SCOTT                          BONUS
SCOTT                          DEPART
SCOTT                          DEPT
SCOTT                          DISCIPLINE
SCOTT                          DISCSPECPROF
SCOTT                          EMP
SCOTT                          EMP1
SCOTT                          NOTE
SCOTT                          PERS
SCOTT                          PLAN_TABLE
SCOTT                          PROFESORI
SCOTT                          SALGRADE
SCOTT                          SALIEX
SCOTT                          STUD1
```

Descr All_tables -- descrie structura tabelei All_tables, care are peste 30 de coloane

```
Name                                      Null?    Type
----------------------------------------- -------- ----------------
OWNER                                     NOT NULL VARCHAR2(30)
TABLE_NAME                                NOT NULL VARCHAR2(30)
TABLESPACE_NAME                                    VARCHAR2(30)
CLUSTER_NAME                                       VARCHAR2(30)
IOT_NAME                                           VARCHAR2(30)
PCT_FREE                                           NUMBER
PCT_USED                                           NUMBER
INI_TRANS                                          NUMBER
MAX_TRANS                                          NUMBER
INITIAL_EXTENT                                     NUMBER
NEXT_EXTENT                                        NUMBER
MIN_EXTENTS                                        NUMBER
MAX_EXTENTS                                        NUMBER
PCT_INCREASE                                       NUMBER
FREELISTS                                          NUMBER
FREELIST_GROUPS                                    NUMBER
LOGGING                                            VARCHAR2(3)
BACKED_UP                                          VARCHAR2(1)
NUM_ROWS                                           NUMBER
BLOCKS                                             NUMBER
EMPTY_BLOCKS                                       NUMBER
AVG_SPACE                                          NUMBER
CHAIN_CNT                                          NUMBER
AVG_ROW_LEN                                        NUMBER
AVG_SPACE_FREELIST_BLOCKS                                  NUMBER
```

Selecturi imbricate

Se admite Select in Select pentru afişare salariaţi care au aceeaşi funcţie ca şi SCOTT.

```
SELECT Ename Nume, Deptno,Job Functie,Sal Salar FROM Emp
      WHERE job=  ANY(SELECT job FROM Emp where Ename='SCOTT');
```

```
NUME       |     DEPTNO | FUNCTIE   |      SALAR
---------- | ---------- | --------- | ----------
FORD       |         20 | ANALYST   |       3000
SCOTT      |         20 | ANALYST   |       3000
```

Dacă comanda SELECT returnează mai mult de o înregistrare se foloseşte ANY sau ALL
ANY – condiţie îndeplinită dacă cel puţin o valoare selectată îndeplineşte condiţia.
ALL - condiţie îndeplinită dacă toate valorile selectate îndeplinesc condiţia.

Dacă trebuie să afişăm toţi salariaţii care au salar mai mare decât salariul mediu din toate departamentele vom scrie:

```
SELECT Ename, Deptno,sal Salar FROM Emp WHERE sal>
      ALL(SELECT avg(sal) FROM Emp GROUP BY deptno);
```

```
ENAME      |     DEPTNO |      SALAR
---------- | ---------- | ----------
JONES      |         20 |       2975
```

```
SCOTT      |          20 |        3000
KING       |          10 |        5000
FORD       |          20 |        3000
```

Dacă schimbăm ALL cu ANY vom obţine salariaţii care au salar mai mare decât salariul mediu din cel puţin un departament.

SELECT Ename, Deptno,sal Salar FROM Emp WHERE sal>
ANY(SELECT avg(sal) FROM Emp GROUP BY deptno)

```
ENAME      |     DEPTNO |      SALAR
---------- | ---------- | ----------
ALLEN      |          30 |        1600
JONES      |          20 |        2975
BLAKE      |          30 |        2850
CLARK      |          10 |        2450
SCOTT      |          20 |        3000
KING       |          10 |        5000
TURNER     |          30 |        1500
FORD       |          20 |        3000
```

1.3.SELF JOIN

Uneori operaţia de Join trebuie făcută între înregistrările aceleeaşi tabele, lucru care se admite deschizând de 2 ori aceeaşi tabelă cu alias-uri diferite.
Fiecare salariat are un şef indicat din motive de normalizare prin codul său Mgr (manager). Dacă vrem ca să afişăm şi date despre şef ele se găsesc în aceeaşi tabelă EMP de salariaţi,

Vom afişa numele şi salariul fiecărui salariat şi pentru fiecare numele si salariul şefului, **deschizând de 2 ori tabela de salariaţi cu aliasul E şi respectiv S** şi făcând un Join. Condiţia de Join va căuta pentru fiecare salariat care este şeful pe baza codului E.Mgr=S.Empno

SELECT E.Ename, E.Sal Salar, S.Ename Nume_sef, S.Sal Salar_sef
FROM EMP E, EMP S WHERE E.Mgr=S.Empno;

```
ENAME      |      SALAR | NUME_SEF   |  SALAR_SEF
---------- | ---------- | ---------- | ----------
FORD       |        3000 | JONES      |        2975
SCOTT      |        3000 | JONES      |        2975
JAMES      |         950 | BLAKE      |        2850
TURNER     |        1500 | BLAKE      |        2850
MARTIN     |        1250 | BLAKE      |        2850
WARD       |        1250 | BLAKE      |        2850
ALLEN      |        1600 | BLAKE      |        2850
MILLER     |        1300 | CLARK      |        2450
ADAMS      |        1100 | SCOTT      |        3000
CLARK      |        2450 | KING       |        5000
BLAKE      |        2850 | KING       |        5000
```

Operaţii pe grup de înregistrări

Funcţiile agregat SUM, COUNT, AVG, MIN, MAX permit calcule asupra înregistrărilor dintr-o tabelă, caz în care în listă de afişare se admit numai aceste funcţii şi se interzice specificarea vreunei coloane a tabelei.

```
SELECT avg(sal) Salar_mediu, Sum(sal) Suma_sal,
       Max(Sal) Sal_max, Min(sal) Min_sal,  Count(*) Nr FROM EMP;
```

```
SALAR_MEDIU |   SUMA_SAL |    SAL_MAX |    MIN_SAL |         NR
----------- | ---------- | ---------- | ---------- | ----------
  1864.0625 |      29825 |       5000 |        200 |         16
```

Atributul GROUP BY se foloseşte pentru a afla date despre un grup de înregistrări specificate prin una sau mai multe coloane care au aceeaşi valoare (facultate, secţie, an, funcţie, salar, oraş, etc.). În acest caz se foloseşte **GROUP BY** si funcţii agregat SUM, COUNT, AVG, MIN, MAX, care se aplică fiecărui grup. Tabela se sortează prima dată după coloana care indică grupul şi apoi se calculează valorile funcţiilor agregat pentru fiecare grup. **Numai coloanele după care se face gruparea pot apare în lista de afişare şi funcţiile agregat.**

Salariile medii şi numărul de salariaţi pe fiecare departament se obţin grupând salariaţii după numărul departamentului. Dacă nu există Group By funcţiile agregat se fac pentru tot fişierul.

```
SELECT E.deptno Nr_dept, dname Nume_Dept ,loc Adresa, avg(sal) Sal_med,
       COUNT(Ename) Nr_salariati    FROM Emp E,Dept D
       WHERE d.deptno=e.deptno
       GROUP BY E.deptno, dname, loc
```

```
   NR_DEPT | NUME_DEPT      | ADRESA        |    SAL_MED | NR_SALARIATI
---------- | -------------- | ------------- | ---------- | ------------
        10 | ACCOUNTING     | NEW YORK      | 2916.66667 |            3
        20 | RESEARCH       | DALLAS        | 1845.83333 |            6
        30 | SALES          | CHICAGO       | 1428.57143 |            7
```

Condiţia de grup se poate verifica cu **HAVING** şi atunci se afişează numai grupurile care îndeplinesc condiţia**.**

Afişare pentru fiecare funcţie suma si media salariilor, salarul maxim, minim si numărul salariaţilor, pentru funcţiile care au salarul mediu > 1000.

```
SELECT Job, avg(sal) Salar_mediu, Sum(sal) Suma_sal,
       Max(Sal) Sal_max, Min(sal) Min_sal,  Count(*) Nr
        FROM EMP GROUP BY job
        HAVING Avg(sal)>1000
```

```
JOB       | SALAR_MEDIU |   SUMA_SAL |    SAL_MAX |    MIN_SAL |         NR
--------- | ----------- | ---------- | ---------- | ---------- | ----------
ANALYST   |        3000 |       6000 |       3000 |       3000 |          2
CLERK     |      1037.5 |       4150 |       1300 |        800 |          4
MANAGER   |  2758.33333 |       8275 |       2975 |       2450 |          3
PRESIDENT |        5000 |       5000 |       5000 |       5000 |          1
SALESMAN  |        1400 |       5600 |       1600 |       1250 |          4
```

1.4.Inner, Self si Outer JOIN

INNER JOIN este Join-ul clasic şi preia din cele două tabele numai înregistrările care au corespondent. Interogarea următoare nu va afişa salariaţii care nu au specificat DeptNo (persoane din conducere,întretinere):

SELECT E.Ename Nume,E.Sal Salar , D.Dname Nume_dept
FROM EMP E, DEPT D WHERE E.Deptno=D.Deptno and sal<1000;

```
NUME             SALAR NUME_DEPT
---------- ---------- ----------
SMITH               800 RESEARCH
JAMES               950 SALES
```

Scrierea standard a unui INNER JOIN este:

SELECT E.Ename Nume, E.Sal Salar , D.Dname Nume_dept
FROM EMP E INNER JOIN DEPT D ON E.Deptno=D.Deptno and sal<1000;

LEFT OUTER JOIN **- ia toate înregistrările din prima tabelă şi**

- Completează coloanele din a doua tabela dacă au corespondent conform condiţiei de Join
- Coloanele din a doua tabelă rămân goale dacă nu există înregistrarea corespondentă

1 SELECT E.Ename Nume, E.Sal Salar, E.deptno, D.Dname Nume_dept
2* FROM EMP E, DEPT D WHERE E.Deptno=D.Deptno(+) and sal<1000

```
NUME             SALAR     DEPTNO NUME_DEPT
---------- ---------- ---------- --------------
SMITH               800         20 RESEARCH
JAMES               950         30 SALES
popa                200
Popescu             600
In comanda anterioara de Inner Join salariatii Popa si Popescu nu
au fost afisati fiindca nu  au un nr de departament specificat.
```

Scrierea standard a unui LEFT OUTER JOIN este:

SELECT E.Ename Nume,E.Sal Salar , D.Dname Nume_dept
FROM EMP E LEFT OUTER JOIN DEPT D
ON E.Deptno=D.Deptno and sal<1000;

RIGHT OUTER JOIN – lucrează asemănător luând toate înregistrările din tabela a doua.

In exemplul următor se afişează toţi salariaţii, dar care nu au telefon o sa aibă coloana TEL si Impuls (număr de impulsuri) necomplectate.

Select P.Nume, P.Adresa, P.Tel, T.Impuls FROM Pers P, Telefon T
Where P.Tel=T.Tel(+);

Semnul (+) se pune după coloana din condiţia de Join corespunzătoare tabelei care este incompletă.

Scrierea standard a unui LEFT OUTER JOIN este:

SELECT E.Ename Nume,E.Sal Salar , D.Dname Nume_dept
FROM EMP E RIGHT OUTER JOIN DEPT D

ON E.Deptno=D.Deptno and sal<1000;

Self JOIN

Este JOIN-ul realizat in cadrul aceleaşi tabele. Un câmp din tabelă referă alt câmp din aceiaşi tabelă. Operaţia se realizează deschizând aceeaşi tabela de 2 ori cu alias-uri diferite.
Se vor afişa date despre salariaţi, numele si salarul şefului folosind tabela EMP, unde fiecare salariat are codul şefului (MGR), care este tot un salariat având codul Empno.

SELECT E.Ename Nume, E.Sal Salar, S.Ename Nume_sef, S.Sal Salar_sef
FROM EMP E, EMP S WHERE E.Mgr=S.Empno;

NUME	SALAR	NUME_SEF	SALAR_SEF	
FORD	3000	JONES	2975	-- Se observă că în lista de salariaţi
SCOTT	3000	JONES	2975	nu există salariatul KING fiindcă
JAMES	950	BLAKE	2850	el este preşedinte şi nu are şef
TURNER	1500	BLAKE	2850	
MARTIN	1250	BLAKE	2850	
WARD	1250	BLAKE	2850	
ALLEN	1600	BLAKE	2850	
MILLER	1300	CLARK	2450	
ADAMS	1100	SCOTT	3000	
CLARK	2450	KING	5000	
BLAKE	2850	KING	5000	
JONES	2975	KING	5000	
SMITH	800	FORD	3000	

Scrierea standard a unui SELF INNER JOIN este:

SELECT E.Ename Nume, E.Sal Salar, S.Ename Nume_sef, S.Sal Salar_sef
FROM EMP E INNER JOIN EMP S ON E.Mgr=S.Empno;

Dacă se utilizează Left Outer Join vor apare toţi salariaţii:

SELECT E.Ename Nume, E.Sal Salar, S.Ename Nume_sef, S.Sal Salar_sef
FROM EMP E, EMP S WHERE E.Mgr=S.Empno(+)

NUME	**SALAR**	**NUME_SEF**	**SALAR_SEF**	
FORD	**3000**	**JONES**	**2975**	
SCOTT	**3000**	**JONES**	**2975**	
JAMES	**950**	**BLAKE**	**2850**	
TURNER	**1500**	**BLAKE**	**2850**	
MARTIN	**1250**	**BLAKE**	**2850**	
WARD	**1250**	**BLAKE**	**2850**	
ALLEN	**1600**	**BLAKE**	**2850**	
MILLER	**1300**	**CLARK**	**2450**	
ADAMS	**1100**	**SCOTT**	**3000**	
CLARK	**2450**	**KING**	**5000**	
BLAKE	**2850**	**KING**	**5000**	
JONES	**2975**	**KING**	**5000**	
SMITH	**800**	**FORD**	**3000**	
KING	**5000**			**-- datele despre şef lipsesc**

1.5.Definire vederi View

Vederile definesc o macheta prin care se vad numai anumite coloane din tabelele bazei de date.
O vedere se defineşte printr-un SELECT care poate conţine unul sau mai multe operaţii de JOIN.
Vederile sunt tratate ca nişte tabele virtuale pentru care nu se alocă spaţiu.
SELECT-ul prin care se defineşte o vedere se ţine intr-un rând din tabela **All_Views** ca o coloana **Tex**t de tip LONG (echivalent cu tipul memo din dBase).
Vederile - simplifică scrierea interogărilor complexe;
- asigura posibilitatea de a acorda drepturi de acces pe vedere si nu pe tabele;
- permite definirea unor tabele virtuale pentru aplicaţii.

CREATE VIEW nume_view (lista_coloane) AS SELECT lista_col
FROM tab1,tab2 WHERE cond [WITH READ ONLY];

CREATE VIEW Vemp (Nume,Salar,Departament) AS
SELECT E.Ename Nume,E.Sal Salar , D.Dname Nume_dept
FROM EMP E, DEPT D WHERE E.Deptno=D.Deptno;

SELECT * FROM Vemp WHERE salar>1200; -- Afişează tabela virtuala executând JOIN.

```
NUME            SALAR DEPARTAMENT
---------- ---------- --------------
MILLER            1300 ACCOUNTING
KING              5000 ACCOUNTING
CLARK             2450 ACCOUNTING
FORD              3000 RESEARCH
SCOTT             3000 RESEARCH
JONES             2975 RESEARCH
TURNER            1500 SALES
BLAKE             2850 SALES
MARTIN            1250 SALES
WARD              1250 SALES
ALLEN             1600 SALES
```

Descr All_Views -- afişează structura tabelei (vederii) cu descrierea vederilor.

```
Name                           Null?    Type
 ----------------------- -------- ---------------
 OWNER                    NOT NULL     VARCHAR2(30)
 VIEW_NAME                NOT NULL     VARCHAR2(30)
 TEXT_LENGTH                                NUMBER
 TEXT                                       LONG
 TYPE_TEXT_LENGTH                           NUMBER
 ...........
```

Afişarea vederilor existente in BD se face din tabela All_Views unde definirea este un Select memorat in coloana Text de tip LONG.
SET LONG 20000 -- setează lungime afişată pentru coloane de tipul LONG

Afişarea vederii Vemp definită anterior se poate face specificând numele în interogarea:

SELECT Owner, View_Name, Text FROM All_Views WHERE View_Name= 'VEMP';

```
OWNER      VIEW_NAME       TEXT
-------------------------- -------------------------- ------------------------------------
SCOTT      VEMP            SELECT E.Ename Nume,E.Sal Salar , D.Dname Nume_dept
                             FROM EMP E, DEPT D WHERE E.Deptno=D.Deptno
```

Vederile simple permit modificările si inserările (DML):

- Definite pe o tabela
- Fără funcţii agregat sau clauza GROUP BY

Vederile complexe conţin operaţii de **JOIN** pe doua sau mai multe tabele sau operaţii de grup. Asupra acestor vederi nu se permit decât operaţii de interogare.

Drepturile de acces pentru grupuri de utilizatori se recomandă să se dea pe vederi si nu pe tabele. Adăugările sau modificări se pot face pe vedere şi fizic se fac în tabela corespunzătoare. Se pot realiza astfel tabele virtuale. Pot exista utilizatori, care au acces la toată tabela şi uni numai la anumite coloane, sau la anumite înregistrări.

La proiectarea BD se va evita fragmentarea tabelelor din motive de " proprietate " sau locaţie, deoarece prelucrările globale se complică. Va exista o singură tabelă pentru fiecare entitate a BD. Intr-o întreprindere va fi **o singură tabelă de stocuri**, chiar dacă sunt 10 magazii şi gestionari. Se va defini câte o vedere Mag_i pentru fiecare magazie şi pentru vederea Mag_i va primi acces nelimitat numai gestionarul $Gest_i$, care va răspunde pentru materialele din magazia lui.

Serviciul de contabilitate sau conducerea va avea acces la toată tabela de Stocuri printr-o vedere **numai la citire,** pentru control şi prelucrări financiar contabile globale. Intr-o comanda se folosesc materiale din diferite magazii. Pentru a calcula valoarea unei comenzi contabilitatea trebuie sa aiba acces la toate materialele indiferent de magazie dar **numai in citire** pentru o vedere **Cont_Stocuri** definita pe tabela Stocuri.

Pentru gestiunea universitară va exista o singură tabelă de **Studenţi** şi mai multe vederi pe facultăţi sau chiar secţii. Fiecare secretară va primi drepturi complete pentru secţia de care răspunde. Conducerea facultăţii va avea drepturi de citire pe **vederea Decan_Studenti** în toate secţiile facultăţii pentru informare, control şi prelucrări statistice. Rectoratul universităţii va avea **drepturi de citire pe vederea R_Studenti** pentru întreaga tabelă de studenţi din toate facultatile. Anumite câmpuri din tabela studenţi pot fi consultate de oricine fiind publice (Nume, adresa, tel, e-mail, data_naşterii,..) si vor fi accesate prin in citire din vederea D_Studenti.

Create View Stud_AC AS Select * From stud Where cods Like 'AC%';
GRANT SELECT, INSERT, UPDATE, DELETE ON Stud_AC TO secr_ac;
GRANT SELECT(Nume, Adresa, Tel) ON Stud TO Public;

Secretara de la secţia AC are drepturi complete asupra studenţilor de la AC.
Câmpurile Nume, Adresa, Tel din fişierul studenţi pot fi citite de oricine dar nu modificate.

Select * From stud_ac Where cods Like 'AC4%'; -- Afişează studenţii din anul 4 AC